

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.



HOW A TEAM STAYS SANE

One change, start to finish.

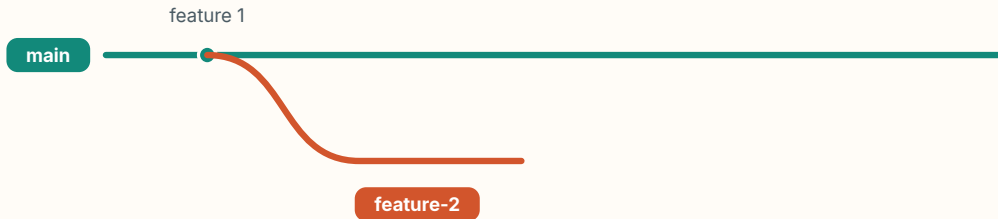
`main` is the version everyone trusts. Here is how a change gets into it.

main



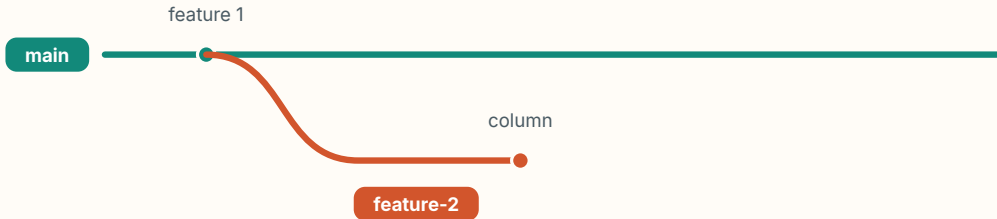
One version, one history, one truth. No folder full of *model_v3_FINAL_jm*.

A branch.



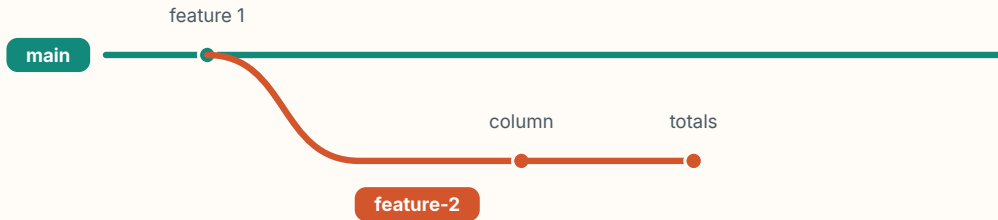
A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

A commit.



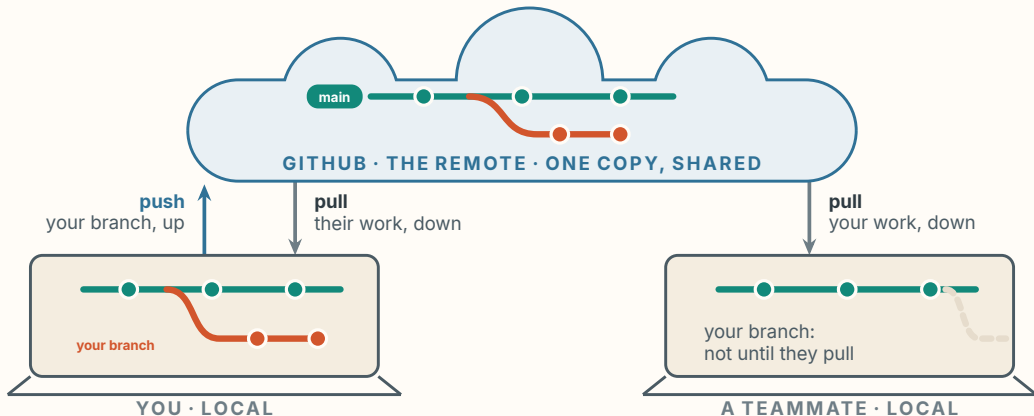
A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

Working on a branch.



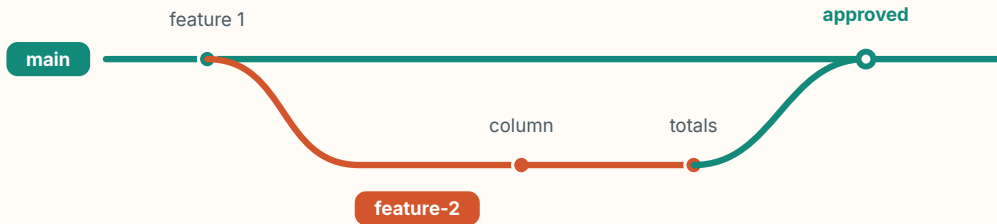
Two commits in and half of it broken. Everyone else's copy is untouched.

Local, and remote.



Until you **push**, your branch is only on your laptop. **Pull** brings their work down. A **pull request** asks someone to read the diff first — today, you.

Merge.



Merge is the sign-off. **main** now has your work, and the next person starts from there.

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

The long run-up.



The ideas were all there. What was missing arrived after 2007: **cheap parallel arithmetic** (graphics cards) and **the web as text**.

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (public crawls from 2011)

Why NVIDIA?



Why is NVIDIA the most valuable company in the world?

One piece of software.

A CPU



a few, fast,
one thing each

A GPU



thousands, slower,
all at once

GAMES · 1999

Millions of pixels,
none waiting for
another. Parallel by
necessity.



CUDA · 2007

Any maths, by any
programmer, on the
games chip.



MINING · 2010s

The same trick:
billions of hashes,
side by side.



TRAINING · 2012 →

AlexNet on two
gaming cards.
Today: thousands
of chips, one model.

The chip was built for games. CUDA let everyone else use it — and training a neural network is nothing but parallel arithmetic.

2018 onwards.



The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**. The next four slides are that second change.

Bottom row, parameters: Radford et al. (2018, 2019), Brown et al. (2020) · GPT-4: an unconfirmed 2023 report (SemiAnalysis) · none published since

Ovid's Unicorn.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

Guess the next word.

1 · HIDE A WORD

The scientist named the population,
after their distinctive _____

2 · GUESS



horn 62%
colour 14%
feature 9%
size 5%

3 · CORRECT

the page says: horn

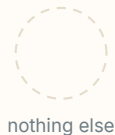
off by a bit?
nudge the numbers

next word, next page · a few billion times

That is all training is: hide the next word, guess, get corrected. Writing is the same game run forwards, one word at a time — and nothing checks it.

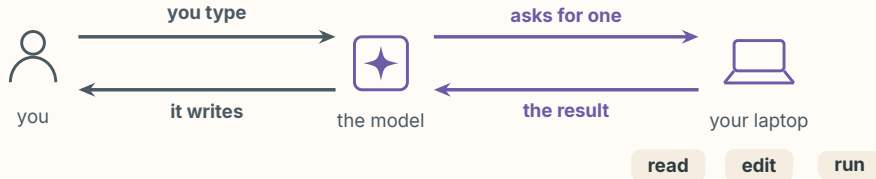
Radford et al. (2019), *Language Models are Unsupervised Multitask Learners* · percentages illustrative

A language model.



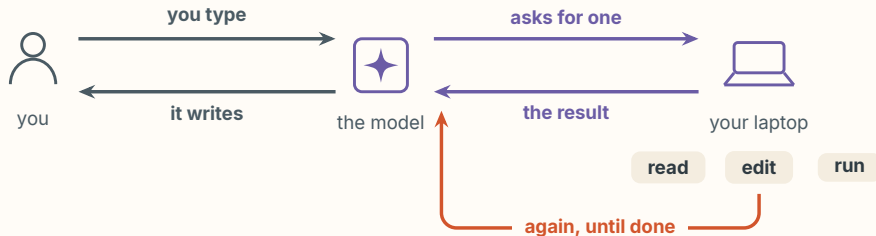
Text in, text out. It has never seen your folder and cannot run anything. In Class 2, **you** were the hands.

Tools.



A **tool** is a named action it may ask for. Three cover nearly everything. The result comes back to **it**, not to you.

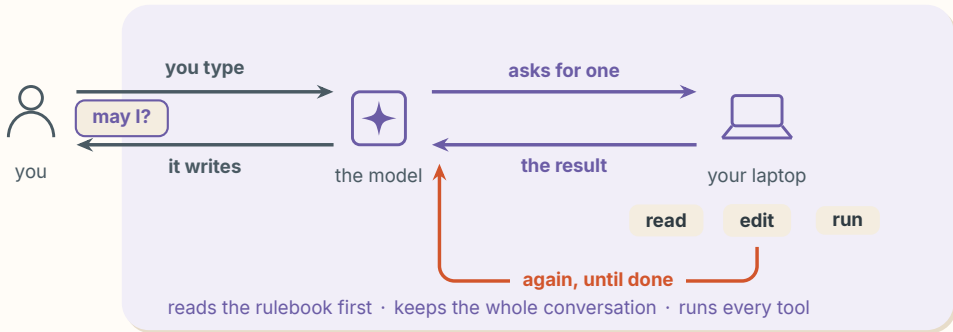
An agent.



Agent = model + tools + a loop. It acts, sees the result, and chooses again.

The harness.

CODEX · THE HARNESS



The software around the model. Codex is one; Claude Code and Cursor are others. Same loop everywhere — **learn it once**.

One exchange, in full.

YOU TYPE what was the last expense?

THE MODEL EMITS `{"tool": "read_file", "path": "data/expenses.csv"}`

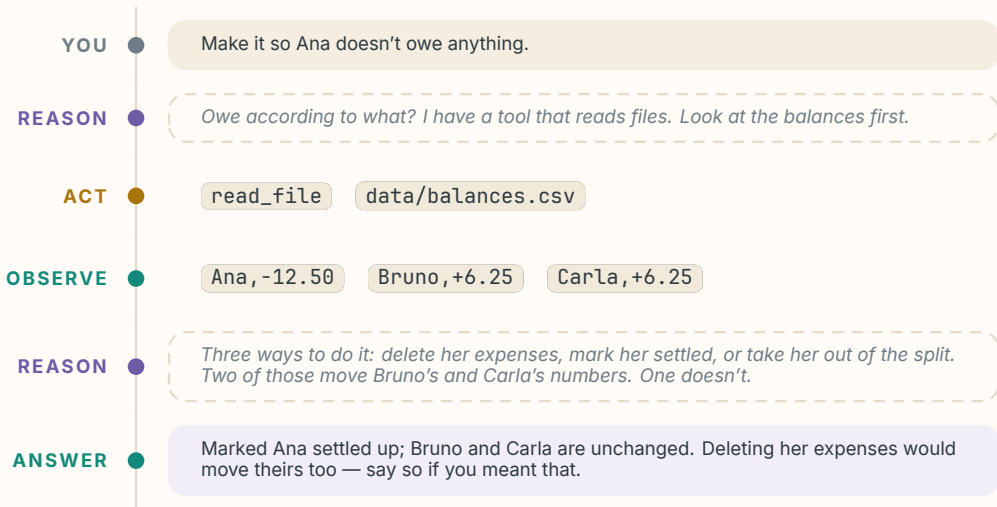
CODEX ASKS *Codex wants to read* `data/expenses.csv` *. Allow?*

CODEX PASTES BACK `29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25`

THE MODEL ANSWERS €30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

Reasoning.



The loop has a name: ReAct.



Reason, act, observe, repeat. Every harness since is this loop with better plumbing.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

Context.

ONE PAGE · EVERYTHING IT CAN SEE

THERE BEFORE THE FIRST THOUGHT

TOOLS

read a file · edit lines · run a command

RULEBOOK

Run the tests before saying done.

YOU

Bug 002: removing a member crashes the app.

ADDED AS IT WORKS, AT THE BOTTOM

IT THINKS

Where is a member removed? I have a tool that reads files.

CALL

read the file that holds the logic

RESULT

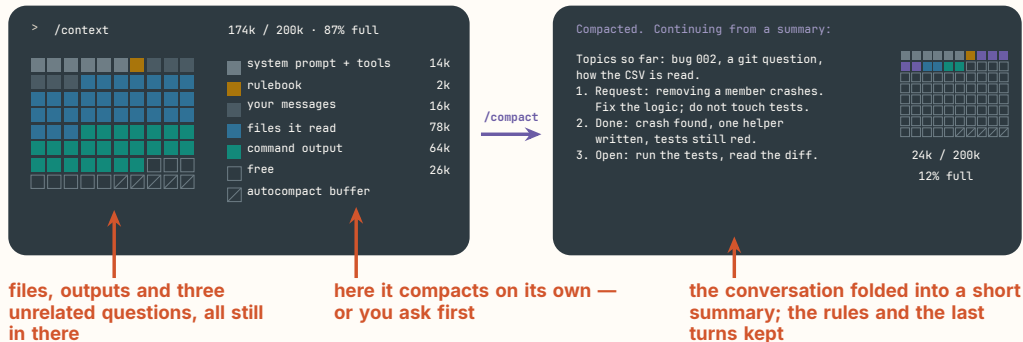
the whole file, 140 lines, now on the page



**the tool's own line is on the page —
that is how it knows which one to call**

Nothing that is not on
the page exists for it.

Context rot.



A full, messy window makes the work worse. Compaction clears it.

Measured on bug-fixing: Rando et al. (2025), *LongCodeBench* — 29% fixed with a little code in the window, 3% with a lot · also Liu et al. (2024), Chroma (2025).

All on the week page.

3

THE CORE OF THE CLASS

Delegating.

Handing work to something fast, tireless, and always sure it is done.

Three questions.

1 · THE BRIEF

How do I brief it?

Once, in writing. It reads it before every task, and never tires of it.

THE RULEBOOK

2 · THE CHECK

How do I know it is done?

Not by asking it. A check written before the work, by someone who is not the agent.

THE TESTS

3 · THE RETRY

And when it is not?

Send the failure back and let it go again. Stop at green, or after twice on the same failure.

THE LOOP

One bug for all three today: removing a member crashes the page.

Say it once.

AGENTS.md

HOW TO RUN

The app, the tests, and how to reset the data: one command each. Things it cannot guess.

BOUNDARIES

- + One branch per spec. Never straight to `main`.
- + Show me the test output, not a summary of it.
- + Never touch the tests. If one looks wrong, say so.

HOW TO REPORT BACK

File and line for every claim. Change code only when asked.

reads it first



every new conversation, before your message.



today: these three lines. Read the diff, commit.

The lines you would otherwise repeat. It reads them before every conversation.

Don't take its word.



WHAT IT SAYS

Done. Removing a member no longer crashes the page, and the tests pass.

✗ **a sentence. Anyone can type it.**

WHAT THE TESTS SAY

```
> run the tests  
test_bug_002.py F  
1 failed, 12 passed
```

✓ **a verdict. When they disagree, this wins.**

WHAT MAKES A CHECK A JUDGE

written before the work

by someone who is not the agent

the same answer every time

A judge is a check that does not need its opinion. Today: the tests from last class.

Red is a to-do list.

```
> run the tests
test_bug_002.py F
___ removing_a_member_removes_their_expenses ___
AssertionError: removing a member must also remove
    their expenses. Write remove_member_expenses and
    call it from remove_member
1 failed, 12 passed
```

3

2

1

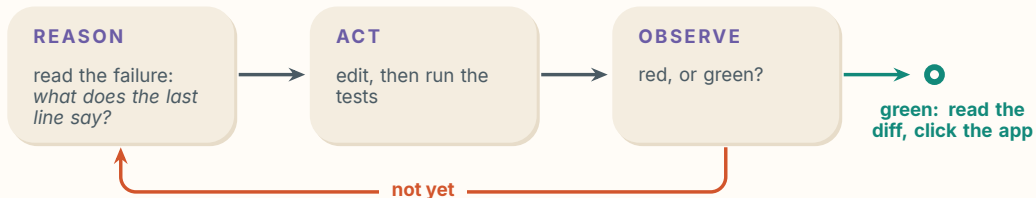
1 how far from done

2 why: that is the to-do, from whoever wrote the test

3 what is checked

Green is the finish line, not the proof: then you read the diff and click the app.

Send the failure back.



Reads its failure, goes again. Stop at green, or after twice on the same failure.

All three, in one message.

1 · THE TASK

Fix bug 002: removing a member crashes the page. Nothing else.

2 · BOUNDARY

On this branch only. Do not change any test.

3 · THE JUDGE

Run the tests, and keep going until every one passes.

4 · EVIDENCE

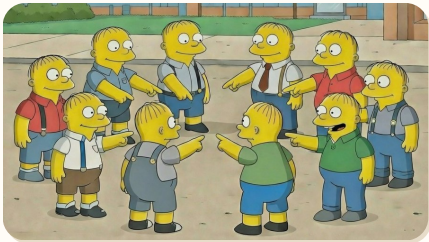
Then paste the full test output and list every file you touched.



One task, the fence from the rulebook, the tests as the finish line, the output as proof. Every loop prompt you will ever write.

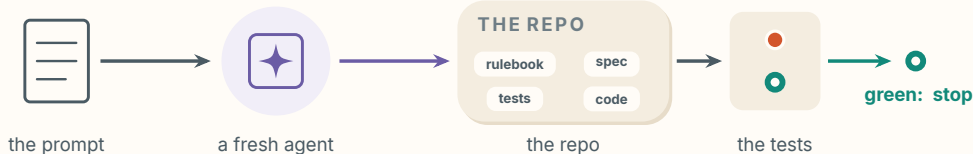
Written out in full on the Week 4 page. Copy it; change two names.

Ralph.



A cartoon character who is cheerfully, relentlessly wrong, and gets there anyway. Not one agent getting better at the task: a crowd of identical beginners, each starting from nothing.

One run.



A file of instructions. A fresh agent reads it, works in the repo, and the tests say red or green. So far, this is the loop from Act 2, with the four parts of an ask in it.

Nobody is watching.

```
while ;; do cat PROMPT.md | claude-code ; done
```



Published in 2025, and that is the entire technique. Every run is a new agent, so the repo is the only memory.

The signs.



THE PROMPT

One task. Full code, no placeholders. What *done* looks like.

WHEN IT FAILS

Don't argue with a run. Put a sign in the prompt, and start again.

THE MEMORY

Spec, plan, rulebook. What it learns goes in a file, or it is gone.

THE STOP

Tests and a commit, every run. A cap on runs. Then you read it all.

/goal

```
/goal every test in tests/ passes, and nothing in tests/ changed
```

2026. The shell loop is now one sentence, in the tool.

WHO DECIDES

A *second*, smaller model reads the conversation after each turn: not yet, done, or impossible.

WHAT IT CAN SEE

Only what the agent put on screen. It cannot run your tests, so the finish line must be something the agent shows.

STILL YOURS

Nobody has automated deciding what *done* means.

Your turn.

- 1 **Branch.** Named after the spec.
- 2 **Red first.** See what is missing before anything is written.
- 3 **New thread. Paste the ask.** Then sit on your hands.
- 4 **Watch it loop.**
- 5 **Green — now don't trust it.** Read the diff. Click the app.
- 6 **Commit, push, pull request, merge.**

Files, test names and the ask: Week 4 page.

SPLITIT

Bug 2, then feature 2

Removing a member crashes the page. Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then conversion by source.

When it goes wrong.

- 1 **It edited a test.** Discard. Say the rule. Run again.
- 2 **"Done" — tests red.** Ask for the output.
- 3 **Same failure, twice.** Stop. Read it yourself. New thread.
- 4 **It fixed three other things.** Discard. One task per loop.
- 5 **Green, app wrong.** Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rules · judge · loop.